EMBEDDED LINUX SYSTEMS

LAB-02

Viết Device Driver & Quản lý File System

Kernel Module | Character Device | sysfs/procfs | MTD/JFFS2 | BusyBox Init

Môn học	Embedded Linux Systems
Mã bài lab	LAB-02
Tên bài lab	Viết Device Driver & Quản lý File System
Thời gian thực hiện	4 giờ (ở nhà 2 giờ - trên lớp 2 giờ)
Chương liên quan	Chapter 8, 9, 10, 11
Kiến trúc mục tiêu	ARMv7-A – Cortex-A9 (QEMU vexpress-a9)
Điều kiện tiên quyết	Hoàn thành LAB-01 (có zImage + initramfs chạy được)
Công cụ yêu cầu	GCC cross-compiler, QEMU, nandsim, mtd-utils, BusyBox
Hình thức nộp bài	File nén + Báo cáo PDF
Hạn nộp bài	[Giảng viên điền]

I. Mục tiêu bài lab

Sau khi hoàn thành bài lab này, sinh viên có khả năng:

- Hiểu và vận dụng được vòng đời của một kernel module: khởi tạo (init), đăng ký, hoạt động, và gỡ bỏ (exit).
- Tự viết một character device driver hoàn chỉnh với các phương thức open, read, write, release theo đúng kernel API.
- Xuất thông tin nội bộ của driver ra userspace thông qua hai giao diện /proc và /sys (sysfs).
- Sử dụng MTD subsystem để mô phỏng flash NAND, phân vùng và thao tác với filesystem JFFS2.
- Tích hợp kernel module vào chuỗi khởi động BusyBox init để module được load tự động khi boot.
- So sánh và lựa chọn filesystem phù hợp (ext4, JFFS2, cramfs) cho các kịch bản embedded khác nhau.

II. Kiến thức nền cần có

2.1. Device Driver Concepts (Chapter 8)

- Ba loại device driver: Character, Block, và Network – sự khác nhau về giao tiếp I/O.
- Cơ chế Major/Minor number: cách kernel ánh xạ /dev node đến driver tương ứng.
- Vòng đời module: module_init() → register_chrdev() → file_operations → module_exit().
- Cấu trúc file_operations: con trỏ hàm cho open, read, write, ioctl, release.
- Truyền dữ liệu an toàn giữa kernel space và user space: copy_to_user(), copy_from_user().
- Module utilities: insmod, rmmod, lsmod, modprobe, modinfo, dmesg.

2.2. File Systems (Chapter 9)

- Khái niệm VFS (Virtual File System): lớp trừu tượng cho phép nhiều loại FS cùng tồn tại.
- ext2/ext3/ext4: journaling, inode, block group – phù hợp cho block device có wear-leveling.
- JFFS2 (Journalling Flash File System v2): thiết kế đặc biệt cho raw NAND/NOR flash, wear-leveling tích hợp.
- cramfs: filesystem nén, read-only, phù hợp cho firmware image tĩnh.
- Pseudo filesystem: procfs (/proc) và sysfs (/sys) – không lưu dữ liệu, phản ánh trạng thái kernel.

2.3. MTD Subsystem (Chapter 10)

- MTD (Memory Technology Device): lớp trừu tượng cho raw flash (NAND, NOR, SPI Flash).
- Sự khác biệt giữa MTD device (/dev/mtdX) và block device (/dev/sdX): MTD không có block abstraction.
- MTD partition: phân vùng flash theo địa chỉ offset và kích thước.
- MTD utilities: flash_erase, nandwrite, nanddump, mtdinfo.

- nandsim: driver mô phỏng NAND flash trong RAM, dùng cho phát triển và test.

2.4. BusyBox Init (Chapter 11)

- BusyBox init và file /etc/inittab: cấu trúc, các action (sysinit, respawn, askfirst...).
- Script /etc/init.d/rcS: nơi thực hiện các lệnh khởi động hệ thống.
- Thứ tự khởi động: kernel exec init → inittab parsing → rcS → shell.

III. Chuẩn bị môi trường

3.1. Điều kiện tiên quyết

> **Yêu cầu:** Bài lab này được xây dựng tiếp trên kết quả của LAB-01. Sinh viên cần có sẵn thư mục ~/embedded_lab1/ với zImage và initramfs đã hoạt động.

Kiểm tra môi trường từ LAB-01 còn đầy đủ:

```
ls ~/embedded_lab1/output/
# Cần có: zImage, vexpress-v2p-ca9.dtb, initramfs.cpio.gz
ls ~/embedded_lab1/rootfs/initramfs/bin/busybox
# Cần có: busybox binary (static linked)
```

```
quocthinh_2303@Prestige-14-AI-Studio:~$ ls -lh ~/embedded_lab1/output/
total 9,8M
-rw-rw-r-- 1 quocthinh_2303 quocthinh_2303 1,1M Thg 9  18 07:44 initramfs.cpio.gz
-rwxrwxr-x 1 quocthinh_2303 quocthinh_2303 4,1M Thg 9  17 22:50 u-boot
-rw-rw-r-- 1 quocthinh_2303 quocthinh_2303  14K Thg 9  17 22:44 vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 quocthinh_2303 quocthinh_2303 4,8M Thg 9  17 22:44 zImage
quocthinh_2303@Prestige-14-AI-Studio:~$ uname -r
6.8.0-138-generic
quocthinh_2303@Prestige-14-AI-Studio:~$ arm-linux-gnueabihf-gcc --version | head -n 1
arm-linux-gnueabihf-gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
quocthinh_2303@Prestige-14-AI-Studio:~$ qemu-system-arm --version | head -n 1
QEMU emulator version 6.2.0 (Debian 1:6.2+dfsg-2ubuntu6.31)
quocthinh_2303@Prestige-14-AI-Studio:~$
```

3.2. Cài đặt công cụ bổ sung

Cài MTD utilities và công cụ filesystem

```
sudo apt install -y mtd-utils
# Bao gồm: flash_erase, nandwrite, nanddump, mkfs.jffs2, mtdinfo
sudo apt install -y squashfs-tools
# mksquashfs, unsquashfs
sudo apt install -y linux-headers-$(uname -r)
# Kernel headers cho host – dùng để build module thử nghiệm trên host
```

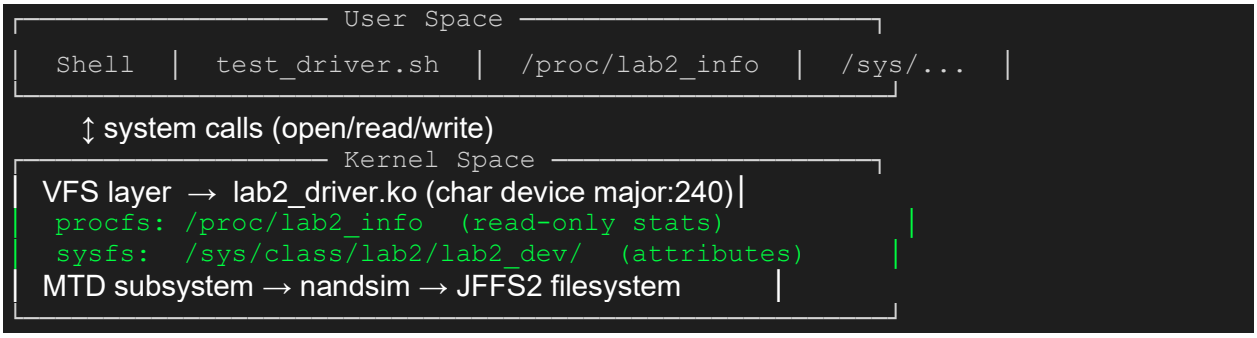
```
quocthinh_2303@Prestige-14-AI-Studio:~$ sudo apt install -y mtd-utils
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mtd-utils is already the newest version (1:2.1.4-1).
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libsmbios-c2
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
quocthinh_2303@Prestige-14-AI-Studio:~$ sudo apt install -y squashfs-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
squashfs-tools is already the newest version (1:4.5-3build1).
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libsmbios-c2
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
quocthinh_2303@Prestige-14-AI-Studio:~$ sudo apt install -y linux-headers-$(uname -r)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
linux-headers-6.8.0-138-generic is already the newest version (6.8.0-138.138~22.04.1).
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libsmbios-c2
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
```

Tạo cấu trúc thư mục LAB-02

```
mkdir -p ~/embedded_lab2/{driver,rootfs,mtd,output,configs}
mkdir -p ~/embedded_lab2/rootfs/initramfs
# Sao chép rootfs từ LAB-01 làm nền
sudo cp -r ~/embedded_lab1/rootfs/initramfs/*
~/embedded_lab2/rootfs/initramfs/
```

```
quocthinh_2303@Prestige-14-AI-Studio:~$ mkdir -p ~/embedded_lab2/{driver,rootfs,mtd,output,configs}
quocthinh_2303@Prestige-14-AI-Studio:~$ mkdir -p ~/embedded_lab2/rootfs/initramfs
quocthinh_2303@Prestige-14-AI-Studio:~$ sudo cp -r ~/embedded_lab1/rootfs/initramfs/* ~/embedded_lab2
/rootfs/initramfs/
quocthinh_2303@Prestige-14-AI-Studio:~$ ls ~/embedded_lab2
configs  driver  mtd  output  rootfs
quocthinh_2303@Prestige-14-AI-Studio:~$ ls ~/embedded_lab2/rootfs/initramfs
bin  dev  etc  init  lib  proc  sbin  sys  tmp  usr
quocthinh_2303@Prestige-14-AI-Studio:~$
```

3.3. Tổng quan kiến trúc bài lab



IV. Hướng dẫn thực hành

PHẦN A – Viết Kernel Module Character Device

> **Mục tiêu:** Viết một character device driver hoàn chỉnh tên lab2_driver, nạp vào kernel, giao tiếp qua /dev/lab2 với các thao tác open, read, write, release

A.1. Cấu trúc Kernel Module cơ bản

Một kernel module tối thiểu cần hai hàm bắt buộc: hàm init được gọi khi insmod và hàm exit được gọi khi rmmod. Với character device, ta thêm vào đó việc đăng ký major number và cấu trúc file_operations.

Tạo file source chính của driver:

```
cd ~/embedded_lab2/driver
cat > lab2_driver.c << 'DRIVER_EOF'
```

Nội dung file lab2_driver.c – Phần 1: Header và định nghĩa

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/slab.h>
#include <linux/mutex.h>
```

```
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Embedded Linux Lab");
MODULE_DESCRIPTION("LAB-02: Simple Character Device Driver");
MODULE_VERSION("1.0");
```

```
#define DEVICE_NAME     "lab2"
#define CLASS_NAME      "lab2_class"
#define BUFFER_SIZE     1024
#define DRIVER_MAJOR    240
/* 240-254: reserved for local/experimental use */
```

Phần 2: Biến toàn cục và bộ đệm nội bộ

```
static int    major_number = DRIVER_MAJOR;
static char   device_buffer[BUFFER_SIZE];
static int    buffer_len = 0;
static int    open_count = 0;
static struct class  *lab2_class  = NULL;
static struct device *lab2_device = NULL;
static DEFINE_MUTEX(lab2_mutex);
```

Phần 3: Triển khai các phương thức file_operations

```
static int lab2_open(struct inode *inode, struct file *file) {
    mutex_lock(&lab2_mutex);
    open_count++;
    mutex_unlock(&lab2_mutex);
    pr_info("lab2_driver: device opened (count=%d)\n", open_count);
    return 0;
}
```

```
static int lab2_release(struct inode *inode, struct file *file) {
    pr_info("lab2_driver: device closed\n");
    return 0;
}
```

```
static ssize_t lab2_read(struct file *file, char __user *buf,
                         size_t len, loff_t *offset) {
    int bytes_read = 0;
    if (*offset >= buffer_len)
        return 0;  /* EOF */
    if (len > buffer_len - *offset)
        len = buffer_len - *offset;
    if (copy_to_user(buf, device_buffer + *offset, len)) {
        pr_err("lab2_driver: copy_to_user failed\n");
        return -EFAULT;
    }
    *offset += len;
    bytes_read = len;
    pr_info("lab2_driver: sent %d bytes to user\n", bytes_read);
    return bytes_read;
}
```

```
static ssize_t lab2_write(struct file *file, const char __user *buf,
                          size_t len, loff_t *offset) {
    if (len > BUFFER_SIZE - 1) {
        pr_warn("lab2_driver: write too large, truncating\n");
        len = BUFFER_SIZE - 1;
    }
    mutex_lock(&lab2_mutex);
    if (copy_from_user(device_buffer, buf, len)) {
        mutex_unlock(&lab2_mutex);
        return -EFAULT;
    }
    buffer_len = len;
    device_buffer[buffer_len] = '\0';
    mutex_unlock(&lab2_mutex);
    pr_info("lab2_driver: received %zu bytes: [%s]\n", len, device_buffer);
    return len;
}
```

Phần 4: Đăng ký file_operations và module init/exit

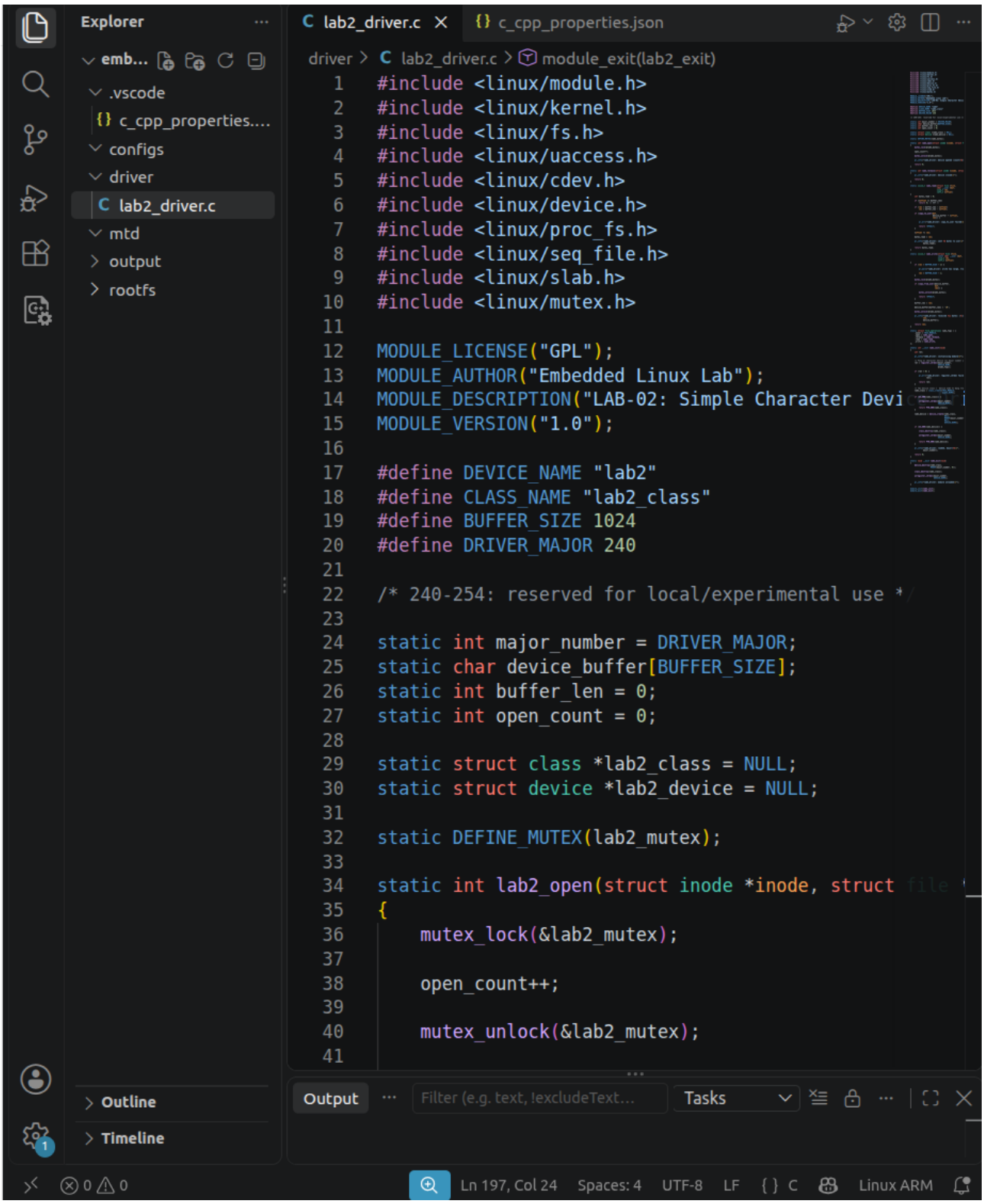
```
static struct file_operations lab2_fops = {
    .owner   = THIS_MODULE,
    .open    = lab2_open,
    .release = lab2_release,
```

```
    .read    = lab2_read,
    .write   = lab2_write,
};
```

```
static int __init lab2_init(void) {
    int ret;
    pr_info("lab2_driver: initializing module\n");
    /* Đăng ký character device với major number cố định */
    ret = register_chrdev(major_number, DEVICE_NAME, &lab2_fops);
    if (ret < 0) {
        pr_err("lab2_driver: register_chrdev failed: %d\n", ret);
        return ret;
    }
    /* Tạo device class và device node tự động trong /sys */
    lab2_class = class_create(THIS_MODULE, CLASS_NAME);
    if (IS_ERR(lab2_class)) {
        unregister_chrdev(major_number, DEVICE_NAME);
        return PTR_ERR(lab2_class);
    }
    lab2_device = device_create(lab2_class, NULL,
                    MKDEV(major_number, 0), NULL, DEVICE_NAME);
    if (IS_ERR(lab2_device)) {
        class_destroy(lab2_class);
        unregister_chrdev(major_number, DEVICE_NAME);
        return PTR_ERR(lab2_device);
    }
    pr_info("lab2_driver: loaded, major=%d\n", major_number);
    return 0;
}
```

```
static void __exit lab2_exit(void) {
    device_destroy(lab2_class, MKDEV(major_number, 0));
    class_destroy(lab2_class);
    unregister_chrdev(major_number, DEVICE_NAME);
    pr_info("lab2_driver: module unloaded\n");
}
```

```
module_init(lab2_init);
module_exit(lab2_exit);
DRIVER_EOF
```




```c
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/slab.h>
#include <linux/mutex.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Embedded Linux Lab");
MODULE_DESCRIPTION("LAB-02: Simple Character Devi
MODULE_VERSION("1.0");

#define DEVICE_NAME "lab2"
#define CLASS_NAME "lab2_class"
#define BUFFER_SIZE 1024
#define DRIVER_MAJOR 240

/* 240-254: reserved for local/experimental use */

static int major_number = DRIVER_MAJOR;
static char device_buffer[BUFFER_SIZE];
static int buffer_len = 0;
static int open_count = 0;

static struct class *lab2_class = NULL;
static struct device *lab2_device = NULL;

static DEFINE_MUTEX(lab2_mutex);

static int lab2_open(struct inode *inode, struct
{
    mutex_lock(&lab2_mutex);

    open_count++;

    mutex_unlock(&lab2_mutex);
```

A.2. Viết Makefile cho Kernel Module

Kernel module phải được build bởi hệ thống build của kernel (KBuild), không phải GCC thông thường. Tạo Makefile sau:

```
cat > Makefile << 'MAKE_EOF'
# Đường dẫn đến kernel source đã build ở LAB-01
KDIR := $(HOME)/embedded_lab1/kernel/linux-5.15
ARCH := arm
```

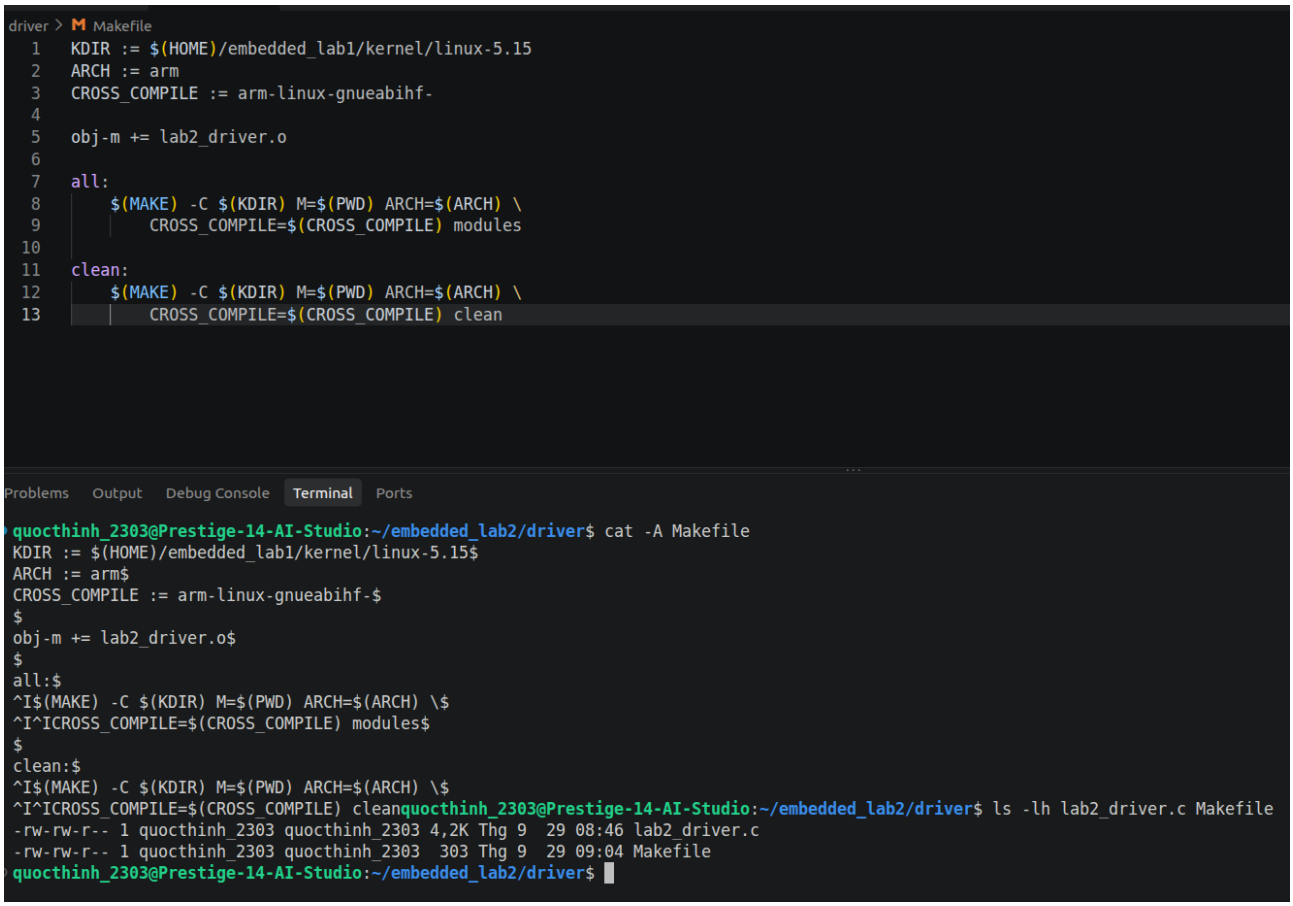
```
CROSS_COMPILE := arm-linux-gnueabihf-

# Tên module (phải khớp với tên file .c)
obj-m += lab2_driver.o

all:
$(MAKE) -C $(KDIR) M=$(PWD) ARCH=$(ARCH) \
        CROSS_COMPILE=$(CROSS_COMPILE) modules

clean:
$(MAKE) -C $(KDIR) M=$(PWD) ARCH=$(ARCH) \
        CROSS_COMPILE=$(CROSS_COMPILE) clean
MAKE_EOF
```

> 📌 **Lưu ý:** Makefile dùng ký tự TAB để thụt đầu dòng lệnh (không dùng space). Nếu copy/paste bị lỗi, hãy dùng lệnh sed -i 's/^ */\t/' Makefile để chuyển đổi.



A.3. Build và kiểm tra Module

Build module cho ARMv7

```
cd ~/embedded_lab2/driver
make
# Nếu build thành công, xuất hiện file: lab2_driver.ko
ls -lh lab2_driver.ko
file lab2_driver.ko
# Mong đợi: ELF 32-bit LSB relocatable, ARM, EABI5
modinfo lab2_driver.ko
```

```
# Xem thông tin: license, author, description, vermagic
```

Kiểm tra vermagic – rất quan trọng

```
modinfo lab2_driver.ko | grep vermagic
# vermagic phải KHỚP CHÍNH XÁC với kernel version trên target
# Ví dụ: 5.15.0 SMP mod_unload ARMv7 p2v8
```

> 📌 **Lưu ý:** vermagic là cơ chế bảo vệ của kernel: module chỉ nạp được vào kernel có cùng version string. Nếu build kernel từ LAB-01 không có thêm local version, vermagic sẽ là '5.15.0'.

```
quocthinh_2303@Prestige-14-AI-Studio:~$ cd ~/embedded_lab2/driver
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ file lab2_driver.ko
lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=b4d557fa41a02a
c79d7426b4f6ff46bb83501ee7, with debug_info, not stripped
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ modinfo lab2_driver.ko
filename:       /home/quocthinh_2303/embedded_lab2/driver/lab2_driver.ko
version:        1.0
description:    LAB-02: Simple Character Device Driver
author:         Embedded Linux Lab
license:        GPL
srcversion:     4E89036A2D2BA07B7DE24B7
depends:
name:           lab2_driver
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ modinfo lab2_driver.ko | grep vermagic
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

Copy module vào rootfs

```
mkdir -p ~/embedded_lab2/rootfs/initramfs/lib/modules/5.15.0
cp lab2_driver.ko ~/embedded_lab2/rootfs/initramfs/lib/modules/5.15.0/
```

A.4. Tạo script test driver trong userspace

Tạo script kiểm tra toàn bộ chức năng của driver từ userspace:

```
cat > ~/embedded_lab2/rootfs/initramfs/usr/bin/test_driver.sh << 'TEST_EOF'
#!/bin/sh
echo '=== LAB-02: Device Driver Test ==='

# Bước 1: Load module
echo '[1] Loading lab2_driver...'
insmod /lib/modules/5.15.0/lab2_driver.ko
if [ $? -ne 0 ]; then echo 'ERROR: insmod failed!'; exit 1; fi

# Bước 2: Xác nhận module đã load
echo '[2] Loaded modules:'
lsmod | grep lab2

# Bước 3: Tạo device node (nếu chưa có)
echo '[3] Creating device node...'
mknod /dev/lab2 c 240 0 2>/dev/null || echo '  (node already exists)'
ls -la /dev/lab2

# Bước 4: Test WRITE
echo '[4] Writing to device...'
```

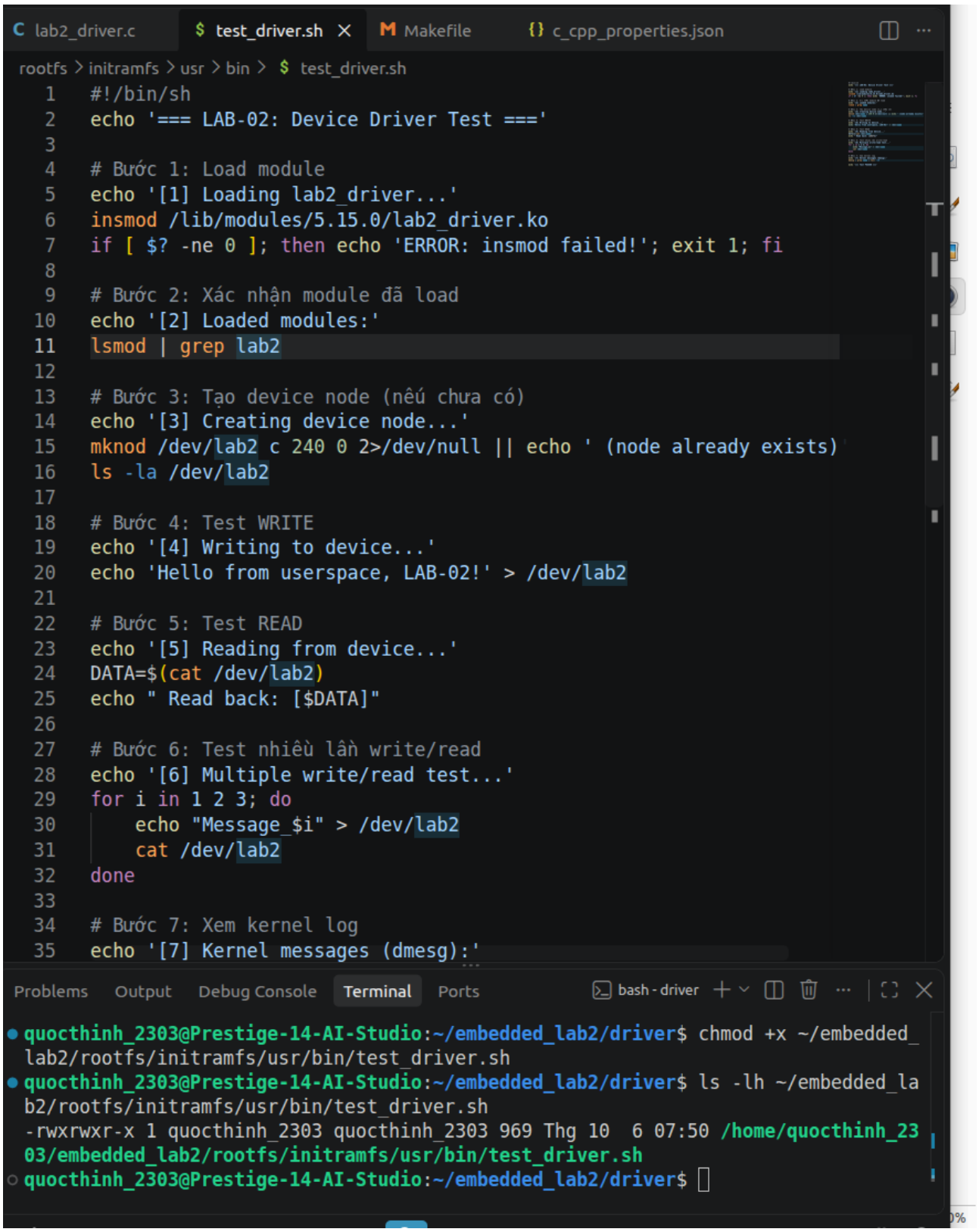
```
echo 'Hello from userspace, LAB-02!' > /dev/lab2

# Bước 5: Test READ
echo '[5] Reading from device...'
DATA=$(cat /dev/lab2)
echo "  Read back: [$DATA]"

# Bước 6: Test nhiều lần write/read
echo '[6] Multiple write/read test...'
for i in 1 2 3; do
    echo "Message_$i" > /dev/lab2
    cat /dev/lab2
done

# Bước 7: Xem kernel log
echo '[7] Kernel messages (dmesg):'
dmesg | grep lab2 | tail -10

echo '=== Test PASSED ==='
TEST_EOF
chmod +x ~/embedded_lab2/rootfs/initramfs/usr/bin/test_driver.sh
```


lab2_driver.c
test_driver.sh
Makefile
c_cpp_properties.json
rootfs > initramfs > usr > bin > test_driver.sh
#!/bin/sh
echo '=== LAB-02: Device Driver Test ==='
Bước 1: Load module
echo '[1] Loading lab2_driver...'
insmod /lib/modules/5.15.0/lab2_driver.ko
if [$? -ne 0]; then echo 'ERROR: insmod failed!'; exit 1; fi
Bước 2: Xác nhận module đã load
echo '[2] Loaded modules:'
lsmod | grep lab2
Bước 3: Tạo device node (nếu chưa có)
echo '[3] Creating device node...'
mknod /dev/lab2 c 240 0 2>/dev/null || echo ' (node already exists)'
ls -la /dev/lab2
Bước 4: Test WRITE
echo '[4] Writing to device...'
echo 'Hello from userspace, LAB-02!' > /dev/lab2
Bước 5: Test READ
echo '[5] Reading from device...'
DATA=$(cat /dev/lab2)
echo " Read back: [$DATA]"
Bước 6: Test nhiều lần write/read
echo '[6] Multiple write/read test...'
for i in 1 2 3; do
echo "Message_$i" > /dev/lab2
cat /dev/lab2
done
Bước 7: Xem kernel log
echo '[7] Kernel messages (dmesg):'
Problems
Output
Debug Console
Terminal
Ports
bash - driver
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ chmod +x ~/embedded_lab2/rootfs/initramfs/usr/bin/test_driver.sh
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ ls -lh ~/embedded_lab2/rootfs/initramfs/usr/bin/test_driver.sh
-rwxrwxr-x 1 quocthinh_2303 quocthinh_2303 969 Thg 10 6 07:50 /home/quocthinh_2303/embedded_lab2/rootfs/initramfs/usr/bin/test_driver.sh
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$

PHẦN B – Giao tiếp qua procfs và sysfs

> **Mục tiêu:** Mở rộng driver để xuất thông tin thống kê ra /proc/lab2_info (procfs) và thuộc tính có thể đọc/ghi ra /sys/class/lab2_class/lab2/ (sysfs)

B.1. Thêm giao diện /proc vào driver

procfs cho phép driver cung cấp thông tin dạng text có thể đọc bằng cat. Ta thêm entry /proc/lab2_info hiển thị thống kê hoạt động của driver.

Thêm các đoạn code sau vào lab2_driver.c (bổ sung sau phần khai báo biến toàn cục):

Thêm vào sau dòng 'static DEFINE_MUTEX(lab2_mutex);'

```
/* ===== PROCFS Implementation ===== */
static struct proc_dir_entry *proc_entry;
```

```
static int lab2_proc_show(struct seq_file *m, void *v) {
    seq_printf(m, "LAB-02 Driver Statistics\n");
    seq_printf(m, "======================\n");
    seq_printf(m, "Driver name   : %s\n", DEVICE_NAME);
    seq_printf(m, "Major number  : %d\n", major_number);
    seq_printf(m, "Buffer size   : %d bytes\n", BUFFER_SIZE);
    seq_printf(m, "Data length   : %d bytes\n", buffer_len);
    seq_printf(m, "Open count    : %d\n", open_count);
    if (buffer_len > 0)
        seq_printf(m, "Last data     : [%s]\n", device_buffer);
    return 0;
}
```

```
static int lab2_proc_open(struct inode *inode, struct file *file) {
    return single_open(file, lab2_proc_show, NULL);
}
```

```
static const struct proc_ops lab2_proc_fops = {
    .proc_open    = lab2_proc_open,
    .proc_read    = seq_read,
    .proc_lseek   = seq_lseek,
    .proc_release = single_release,
};
```

```
static DEFINE_MUTEX(lab2_mutex);
/* ===== PROCFS Implementation ===== */
static struct proc_dir_entry *proc_entry;

static int lab2_proc_show(struct seq_file *m, void *v) {
    seq_printf(m, "LAB-02 Driver Statistics\n");
    seq_printf(m, "========================\n");
    seq_printf(m, "Driver name : %s\n", DEVICE_NAME);
    seq_printf(m, "Major number : %d\n", major_number);
    seq_printf(m, "Buffer size : %d bytes\n", BUFFER_SIZE);
    seq_printf(m, "Data length : %d bytes\n", buffer_len);
    seq_printf(m, "Open count : %d\n", open_count);

    if (buffer_len > 0)
        seq_printf(m, "Last data : [%s]\n", device_buffer);

    return 0;
}

static int lab2_proc_open(struct inode *inode, struct file *file) {
    return single_open(file, lab2_proc_show, NULL);
}

static const struct proc_ops lab2_proc_fops = {
    .proc_open = lab2_proc_open,
    .proc_read = seq_read,
    .proc_lseek = seq_lseek,
    .proc_release = single_release,
};
```

Thêm vào cuối hàm lab2_init() – trước dòng return 0

```
/* Tạo /proc/lab2_info */
    proc_entry = proc_create("lab2_info", 0444, NULL, &lab2_proc_fops);
    if (!proc_entry)
        pr_warn("lab2_driver: failed to create /proc/lab2_info\n");
    else
        pr_info("lab2_driver: /proc/lab2_info created\n");
```

```
    pr_info("lab2_driver: loaded, major=%d\n",
            major_number);
    /* Tạo /proc/lab2_info */
    proc_entry = proc_create("lab2_info", 0444, NULL, &lab2_proc_fops);
    if (!proc_entry)
        pr_warn("lab2_driver: failed to create /proc/lab2_info\n");
    else
        pr_info("lab2_driver: /proc/lab2_info created\n");

    return 0;
}
```

Thêm vào đầu hàm lab2_exit() – trước device_destroy

```
    if (proc_entry)
        proc_remove(proc_entry);
```

```
static void __exit lab2_exit(void)
{
    if (proc_entry)
        proc_remove(proc_entry);
    device_destroy(lab2_class,
                   MKDEV(major_number, 0));

    class_destroy(lab2_class);
```

B.2. Thêm giao diện sysfs vào driver

sysfs cho phép export từng thuộc tính của driver thành các file riêng biệt trong /sys. Không giống procfs, sysfs có cấu trúc phân cấp và hỗ trợ cả đọc lẫn ghi.

Thêm sysfs attribute 'buffer_len' (read-only)

```
/* ===== SYSFS Implementation ===== */
static ssize_t buffer_len_show(struct device *dev,
                struct device_attribute *attr, char *buf) {
    return sprintf(buf, "%d\n", buffer_len);
}
```

```
static ssize_t open_count_show(struct device *dev,
                struct device_attribute *attr, char *buf) {
    return sprintf(buf, "%d\n", open_count);
}
```

```
static ssize_t last_data_show(struct device *dev,
                struct device_attribute *attr, char *buf) {
    if (buffer_len > 0)
        return sprintf(buf, "%s\n", device_buffer);
    return sprintf(buf, "(empty)\n");
}
```

```
static DEVICE_ATTR_RO(buffer_len);
static DEVICE_ATTR_RO(open_count);
static DEVICE_ATTR_RO(last_data);
```

```
static struct attribute *lab2_attrs[] = {
    &dev_attr_buffer_len.attr,
    &dev_attr_open_count.attr,
    &dev_attr_last_data.attr,
    NULL,
};
```

```
static struct attribute_group lab2_attr_group = {
    .attrs = lab2_attrs,
};
```

```c
/* ===== SYSFS Implementation */
static ssize_t buffer_len_show(struct device *dev, struct device_attribute *attr, char *buf) {
    return sprintf(buf, "%d\n", buffer_len);
}

static ssize_t open_count_show(struct device *dev, struct device_attribute *attr, char *buf) {
    return sprintf(buf, "%d\n", open_count);
}

static ssize_t last_data_show(struct device *dev, struct device_attribute *attr, char *buf) {
    if (buffer_len > 0)
        return sprintf(buf, "%s\n", device_buffer);
    return sprintf(buf, "(empty)\n");
}

static DEVICE_ATTR_RO(buffer_len);
static DEVICE_ATTR_RO(open_count);
static DEVICE_ATTR_RO(last_data);

static struct attribute *lab2_attrs[] = {
    &dev_attr_buffer_len.attr,
    &dev_attr_open_count.attr,
    &dev_attr_last_data.attr,
    NULL,
};

static struct attribute_group lab2_attr_group = {
    .attrs = lab2_attrs,
};
```

Thêm vào lab2_init() sau device_create thành công

```c
    /* Đăng ký sysfs attribute group */
    ret = sysfs_create_group(&lab2_device->kobj, &lab2_attr_group);
    if (ret)
        pr_warn("lab2_driver: sysfs_create_group failed\n");
```

```c
/* Dang ky sysfs attribute group */
ret = sysfs_create_group(&lab2_device->kobj, &lab2_attr_group);
if (ret)
    pr_warn("lab2_driver: sysfs_create_group failed\n");
```

Thêm vào lab2_exit() trước device_destroy

```c
    sysfs_remove_group(&lab2_device->kobj, &lab2_attr_group);
```

```c
static void __exit lab2_exit(void)
{
    if (proc_entry)
        proc_remove(proc_entry);
    sysfs_remove_group(&lab2_device->kobj, &lab2_attr_group);
    device_destroy(lab2_class, MKDEV(major_number, 0));

    class_destroy(lab2_class);

    unregister_chrdev(major_number, DEVICE_NAME);

    pr_info("lab2_driver: module unloaded\n");
}
```

B.3. Build lại và kiểm tra procfs/sysfs

```
cd ~/embedded_lab2/driver
make clean && make
cp lab2_driver.ko ~/embedded_lab2/rootfs/initramfs/lib/modules/5.15.0/
```

Tạo script kiểm tra /proc và /sys:

```
cat > ~/embedded_lab2/rootfs/initramfs/usr/bin/test_procfs.sh << 'PROC_EOF'
#!/bin/sh
echo '=== Test procfs & sysfs ==='
insmod /lib/modules/5.15.0/lab2_driver.ko
mknod /dev/lab2 c 240 0 2>/dev/null

# Test procfs
echo '--- /proc/lab2_info (before write) ---'
cat /proc/lab2_info

# Ghi dữ liệu vào device
echo 'Embedded Linux Lab2 Test Data' > /dev/lab2

echo '--- /proc/lab2_info (after write) ---'
cat /proc/lab2_info

# Test sysfs
SYSFS=/sys/class/lab2_class/lab2
echo '--- sysfs attributes ---'
echo "buffer_len : $(cat $SYSFS/buffer_len)"
echo "open_count : $(cat $SYSFS/open_count)"
echo "last_data  : $(cat $SYSFS/last_data)"
echo '=== procfs/sysfs Test PASSED ==='
PROC_EOF
chmod +x ~/embedded_lab2/rootfs/initramfs/usr/bin/test_procfs.sh
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ chmod +x ~/embedded_lab2/rootfs/initramfs/usr/bin/test_procfs.sh
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ ls -lh ~/embedded_lab2/rootfs/initramfs/usr/bin/test_procfs.sh
-rwxrwxr-x 1 quocthinh_2303 quocthinh_2303 598 Thg 10  8 22:04 /home/quocthinh_2303/embedded_lab2/rootfs/initramfs/usr/bin/test_procfs.sh
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat ~/embedded_lab2/rootfs/initramfs/usr/bin/test_procfs.sh
#!/bin/sh
echo '=== Test procfs & sysfs ==='
insmod /lib/modules/5.15.0/lab2_driver.ko
mknod /dev/lab2 c 240 0 2>/dev/null

# Test procfs
echo '--- /proc/lab2_info (before write) ---'
cat /proc/lab2_info

#Ghi du lieu vao device
echo 'Embedded Linux Lab2 Test Data' > /dev/lab2

echo '--- /proc/lab2_info (after write) ---'
cat /proc/lab2_info

#Test sysfs
SYSFS=/sys/class/lab2_class/lab2
echo '--- sysfs attributes ---'
echo "buffer_len : $(cat $SYSFS/buffer_len)"
echo "open_count : $(cat $SYSFS/open_count)"
echo "last_data  : $(cat $SYSFS/last_data)"

echo '=== procfs/sysfs/ Test PASSED ==='
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

PHẦN C – MTD Subsystem và JFFS2 File System

> **Mục tiêu:** Sử dụng nandsim để mô phỏng NAND flash, phân vùng MTD, format JFFS2, mount và kiểm tra tính bền vững (persistence) của dữ liệu

C.1. Tổng quan về MTD và JFFS2

Trong hệ thống nhúng, raw flash (NAND/NOR) không thể sử dụng trực tiếp với ext4 vì:

- Flash không hỗ trợ overwrite – phải xóa (erase) trước khi ghi lại.
- Erase chỉ thực hiện theo đơn vị erase block (thường 128KB hoặc 256KB).
- Flash có giới hạn số lần erase (~100.000 chu kỳ) – cần wear leveling.
- NAND flash có thể có bad block từ nhà sản xuất – cần bad block management.

JFFS2 giải quyết tất cả vấn đề trên: nó là log-structured filesystem, tự xử lý wear leveling và bad block, và hỗ trợ compression để tiết kiệm không gian flash.

So sánh các filesystem phù hợp cho hệ thống nhúng:

Filesystem	Phù hợp cho	Không phù hợp cho
ext4	eMMC, SD card, HDD (có block abstraction)	Raw NAND/NOR flash
JFFS2	Raw NAND/NOR flash, read/write workload	Filesystem lớn (mount chậm do full scan)
cramfs	Read-only firmware image, ROM	Dữ liệu cần ghi hoặc sửa đổi
UBIFS	Large NAND flash, performance critical	Nhỏ hơn 4MB hoặc NOR flash nhỏ
SquashFS	Compressed read-only rootfs	Dữ liệu runtime cần thay đổi

C.2. Kích hoạt nandsim và kiểm tra MTD trên Host

Phần C thực hiện trên máy host Ubuntu (không cần QEMU) vì nandsim là module của host kernel và cần quyền root để thao tác với /dev/mtd.

> **⚠ Cảnh báo:** Các lệnh trong phần C.2 đến C.5 chạy trên HOST Ubuntu với quyền sudo, không phải trong QEMU. Ghi rõ điều này trong báo cáo.

Bước 1: Load module nandsim

```
sudo modprobe nandsim
# nandsim tạo NAND flash ảo trong RAM
sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0x35
# 0x20=ST Micro, 0x35=32MB NAND với 512B page, 16KB erase block
sudo modprobe mtdblock
sudo modprobe jffs2
cat /proc/mtd
```

```
# xác nhận device
ls /dev/mtdblock*
# xác nhận block interface có sẵn
ls /dev/mtd*
# Mong đợi: /dev/mtd0  /dev/mtd0ro
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo modprobe nandsim first_id_byte=0x20 sec
ond_id_byte=0x35
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo modprobe mtdblock
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo modprobe jffs2
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /proc/mtd
dev:    size   erasesize  name
mtd0: 02000000 00001000 "BIOS"
mtd1: 02000000 00004000 "NAND simulator partition 0"
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ ls /dev/mtd*
/dev/mtd0  /dev/mtd0ro  /dev/mtd1  /dev/mtd1ro  /dev/mtdblock0  /dev/mtdblock1
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

Bước 2: Xem thông tin MTD device

```
sudo mtdinfo /dev/mtd0
# Hiển thị: Name, Type, Erase size, Write size, Size, Flags
cat /proc/mtd
# Liệt kê tất cả MTD devices hiện có
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo mtdinfo /dev/mtd1
mtd1
Name:                           NAND simulator partition 0
Type:                           nand
Eraseblock size:                16384 bytes, 16.0 KiB
Amount of eraseblocks:          2048 (33554432 bytes, 32.0 MiB)
Minimum input/output unit size: 512 bytes
Sub-page size:                  256 bytes
OOB size:                       16 bytes
Character device major/minor:   90:2
Bad blocks are allowed:         true
Device is writable:             true

quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /proc/mtd
dev:    size   erasesize  name
mtd0: 02000000 00001000 "BIOS"
mtd1: 02000000 00004000 "NAND simulator partition 0"
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

C.3. Xóa và Format JFFS2

Bước 1: Xóa toàn bộ flash (bắt buộc trước khi ghi)

```
sudo flash_erase /dev/mtd0 0 0
# Tham số: device, offset, số erase block (0 = tất cả)
# Sau lệnh này, toàn bộ MTD được điền 0xFF (trạng thái erase)
# lưu ý dùng : cat /proc/mtd để xác định đúng mtd nào là của "NAND simulator
partition 0"
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo flash_erase /dev/mtd1 0 0
Erasing 32768 Kibyte @ 0 -- 100 % complete
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

Bước 2: Tạo JFFS2 image từ thư mục mẫu

```
mkdir -p /tmp/jffs2_data
echo 'Hello from JFFS2!' > /tmp/jffs2_data/hello.txt
echo 'Embedded Linux Lab2' > /tmp/jffs2_data/info.txt
mkdir -p /tmp/jffs2_data/logs
date > /tmp/jffs2_data/logs/boot.log
```

```
sudo mkfs.jffs2 \
    --root=/tmp/jffs2_data \
    --eraseblock=0x4000 \
  # 0x4000 = 16KB: khớp erase size của nandsim
    --output=/tmp/rootfs.jffs2 \
    --pad \
    # Pad đến cuối erase block
    --no-cleanmarkers
  # Cần cho NAND (JFFS2 cleanmarker không dùng OOB)
ls -lh /tmp/rootfs.jffs2
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ mkdir -p /tmp/jffs2_data
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ echo 'Hello from JFFS2!' > /tmp/jffs2_data/h
ello.txt
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ echo 'Embedded Linux Lab2' > /tmp/jffs2_data
/info.txt
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ mkdir -p /tmp/jffs2_data/logs
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ date > /tmp/jffs2_data/logs/boot.log
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo mkfs.jffs2 \
  --root=/tmp/jffs2_data \
  --eraseblock=0x4000 \
  --output=/tmp/rootfs.jffs2 \
  --pad \
  --no-cleanmarkers
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ ls -lh /tmp/rootfs.jffs2
-rw-r--r-- 1 root root 16K Thg 10  8 23:21 /tmp/rootfs.jffs2
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ ls -R /tmp/jffs2_data
/tmp/jffs2_data:
hello.txt  info.txt  logs

/tmp/jffs2_data/logs:
boot.log
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

Bước 3: Ghi JFFS2 image vào MTD

```
sudo nandwrite -p /dev/mtd0 /tmp/rootfs.jffs2
# -p: pad nếu dữ liệu không đủ page size
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /proc/mtd
dev:    size   erasesize  name
mtd0: 02000000 00001000 "BIOS"
mtd1: 02000000 00004000 "NAND simulator partition 0"
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo nandwrite -p /dev/mtd1 /tmp/rootfs.jffs
2
Writing data to block 0 at offset 0x0
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

C.4. Mount và thao tác với JFFS2

```
sudo mkdir -p /mnt/jffs2
sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
# mtdblock0: block device interface của mtd0 (qua MTD block emulation)
```

Kiểm tra nội dung đã ghi

```
ls -la /mnt/jffs2/
cat /mnt/jffs2/hello.txt
cat /mnt/jffs2/info.txt
cat /mnt/jffs2/logs/boot.log
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo mkdir -p /mnt/jffs2
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo mount -t jffs2 /dev/mtdblock1 /mnt/jffs
2
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ ls -la /mnt/jffs2/
total 5
drwxr-xr-x 4 root           root              0 Thg 1   1  1970 .
drwxr-xr-x 3 root           root           4096 Thg 10  8 23:24 ..
-rw-rw-r-- 1 quocthinh_2303 quocthinh_2303   18 Thg 10  8 23:21 hello.txt
-rw-rw-r-- 1 quocthinh_2303 quocthinh_2303   20 Thg 10  8 23:21 info.txt
drwxrwxr-x 2 quocthinh_2303 quocthinh_2303    0 Thg 10  8 23:21 logs
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /mnt/jffs2/hello.txt
Hello from JFFS2!
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /mnt/jffs2/info.txt
Embedded Linux Lab2
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /mnt/jffs2/logs/boot.log
Thứ năm, 08 Tháng 10 năm 2026 23:21:19 +07
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

Ghi thêm dữ liệu vào JFFS2 (kiểm tra write)

```
sudo sh -c 'echo "Written at $(date)" >> /mnt/jffs2/logs/boot.log'
sudo sh -c 'echo "New runtime file" > /mnt/jffs2/runtime.txt'
cat /mnt/jffs2/logs/boot.log
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo sh -c 'echo "Written at $(date)" >> /mn
t/jffs2/logs/boot.log'
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo sh -c 'echo "New runtime file" > /mnt/j
ffs2/runtime.txt'
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /mnt/jffs2/logs/boot.log
Thứ năm, 08 Tháng 10 năm 2026 23:21:19 +07
Written at Thứ năm, 08 Tháng 10 năm 2026 23:26:50 +07
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

Kiểm tra persistence: umount rồi mount lại

```
sudo umount /mnt/jffs2
sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
cat /mnt/jffs2/logs/boot.log
# Dữ liệu ghi trước đó phải còn nguyên sau khi umount/mount lại
cat /mnt/jffs2/runtime.txt
sudo umount /mnt/jffs2
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo umount /mnt/jffs2
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo mount -t jffs2 /dev/mtdblock1 /mnt/jffs
2
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /mnt/jffs2/logs/boot.log
Thứ năm, 08 Tháng 10 năm 2026 23:21:19 +07
Written at Thứ năm, 08 Tháng 10 năm 2026 23:26:50 +07
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /mnt/jffs2/runtime.txt
New runtime file
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo umount /mnt/jffs2
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

> 📌 **Lưu ý:** Ghi lại toàn bộ output của các lệnh trên. Đây là bằng chứng quan trọng cho thấy JFFS2 có persistence – dữ liệu không bị mất khi umount.

C.5. Kiểm tra và dump MTD raw data

```
sudo nanddump -o -l 512 /dev/mtd0 /tmp/dump.bin
# -o: bao gồm OOB data, -l 512: dump 512 bytes đầu tiên
hexdump -C /tmp/dump.bin | head -20
# Xem header của JFFS2 image: magic bytes 0x85 0x19
```

Cleanup – gỡ nandsim

```
sudo rmmod nandsim
ls /dev/mtd* 2>/dev/null || echo 'MTD devices removed'
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo nanddump --oob -l 512 -f /tmp/dump.bin
/dev/mtd1
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 16384, page size 512, OOB size 16
Dumping data starting at 0x00000000 and ending at 0x00000200...
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ ls -lh /tmp/dump.bin
-rw-r--r-- 1 root root 528 Thg 10  8 23:30 /tmp/dump.bin
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ hexdump -C /tmp/dump.bin | head -20
00000000  85 19 01 e0 31 00 00 00  1d d9 82 42 01 00 00 00  |....1......B....|
00000010  00 00 00 00 02 00 00 00  f3 c2 c7 6a 09 08 00 00  |...........j....|
00000020  00 67 4e 0f 15 da 69 f4  68 65 6c 6c 6f 2e 74 78  |.gN...i.hello.tx|
00000030  74 ff ff ff 85 19 02 e0  56 00 00 00 09 64 e7 62  |t.......V....d.b|
00000040  02 00 00 00 01 00 00 00  b4 81 00 00 e8 03 e8 03  |................|
00000050  12 00 00 00 f3 c2 c7 6a  f3 c2 c7 6a f3 c2 c7 6a  |.......j...j...j|
00000060  00 00 00 00 12 00 00 00  12 00 00 00 00 00 00 00  |................|
00000070  ee dd 4b b3 cc c0 76 c8  48 65 6c 6c 6f 20 66 72  |..K...v.Hello fr|
00000080  6f 6d 20 4a 46 46 53 32  21 0a ff ff 85 19 01 e0  |om JFFS2!.......|
00000090  30 00 00 00 78 be 3e fa  01 00 00 00 01 00 00 00  |0...x.>.........|
000000a0  03 00 00 00 f7 c2 c7 6a  08 08 00 00 61 d3 75 06  |.......j....a.u.|
000000b0  a9 de 79 72 69 6e 66 6f  2e 74 78 74 85 19 02 e0  |..yrinfo.txt....|
000000c0  58 00 00 00 3a 13 38 82  03 00 00 00 01 00 00 00  |X...:.8.........|
000000d0  b4 81 00 00 e8 03 e8 03  14 00 00 00 f7 c2 c7 6a  |...............j|
000000e0  f7 c2 c7 6a f7 c2 c7 6a  00 00 00 00 14 00 00 00  |...j...j........|
000000f0  14 00 00 00 00 00 00 00  8a 74 43 dd 3a 16 e6 b4  |.........tC.:...|
00000100  45 6d 62 65 64 64 65 64  20 4c 69 6e 75 78 20 4c  |Embedded Linux L|
00000110  61 62 32 0a 85 19 01 e0  2c 00 00 00 5f 56 f1 e0  |ab2.....,..._V..|
00000120  01 00 00 00 02 00 00 00  04 00 00 00 0c c2 c7 6a  |...............j|
00000130  04 04 00 00 dc c0 e8 ba  40 19 cb d1 6c 6f 67 73  |........@...logs|
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo umount /mnt/jffs2
umount: /mnt/jffs2: not mounted.
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo modprobe -r mtdblock
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ sudo modprobe -r nandsim
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ lsmod | grep nandsim
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$ cat /proc/mtd
dev:    size   erasesize  name
mtd0: 02000000 00001000 "BIOS"
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/driver$
```

PHẦN D – Tích hợp BusyBox Init & Auto-load Module

> **Mục tiêu:** Cập nhật rootfs để kernel module lab2_driver.ko được load tự động trong quá trình boot qua BusyBox init script, sau đó build lại initramfs và boot thành công trên QEMU

D.1. Cập nhật init script để auto-load module

BusyBox init đọc file /etc/inittab khi khởi động và thực thi script /etc/init.d/rcS trong action sysinit. Ta thêm lệnh insmod vào rcS để module được load tự động.

Cập nhật /etc/init.d/rcS

```
cat > ~/embedded_lab2/rootfs/initramfs/etc/init.d/rcS << 'RCS_EOF'
#!/bin/sh
echo '[ LAB-02 ] Starting system initialization...'

# Mount essential pseudo filesystems
mount -t proc none /proc
mount -t sysfs none /sys
mount -t tmpfs none /tmp
mount -t devtmpfs none /dev 2>/dev/null || true

# Populate /dev nếu devtmpfs không available
[ -c /dev/console ] || mknod /dev/console c 5 1
[ -c /dev/null    ] || mknod /dev/null c 1 3
[ -c /dev/zero    ] || mknod /dev/zero c 1 5

# Set hostname
hostname embedded-lab2
echo embedded-lab2 > /proc/sys/kernel/hostname

# ═══ AUTO-LOAD KERNEL MODULE ═══
echo '[ LAB-02 ] Loading lab2_driver module...'
if [ -f /lib/modules/5.15.0/lab2_driver.ko ]; then
    insmod /lib/modules/5.15.0/lab2_driver.ko
    if [ $? -eq 0 ]; then
        echo '[ LAB-02 ] lab2_driver loaded OK'
    # Tạo device node ngay sau khi module load
        mknod /dev/lab2 c 240 0
        chmod 666 /dev/lab2
        echo '[ LAB-02 ] /dev/lab2 created (major=240, minor=0)'
    else
        echo '[ LAB-02 ] ERROR: insmod lab2_driver failed!'
    fi
else
    echo '[ LAB-02 ] WARNING: lab2_driver.ko not found'
fi

echo '[ LAB-02 ] System initialization complete!'
echo '  -> Run test_driver.sh  to test character device'
echo '  -> Run test_procfs.sh  to test proc/sysfs interfaces'
echo '  -> cat /proc/lab2_info to view driver stats'
RCS_EOF
```

```
chmod +x ~/embedded_lab2/rootfs/initramfs/etc/init.d/rcS
```

```
quocthinh_2303@Prestige-14-AI-Studio:~$ code ~/embedded_lab2/rootfs/initramfs/etc/init.d/rcS
quocthinh_2303@Prestige-14-AI-Studio:~$ chmod +x ~/embedded_lab2/rootfs/initramfs/etc/init.d/rcS
quocthinh_2303@Prestige-14-AI-Studio:~$ ls -lh ~/embedded_lab2/rootfs/initramfs/etc/init.d/rcS
-rwxr-xr-x 1 quocthinh_2303 quocthinh_2303 1,3K Thg 10  9 06:42 /home/quocthinh_2303/embedded_lab2/ro
otfs/initramfs/etc/init.d/rcS
quocthinh_2303@Prestige-14-AI-Studio:~$ grep -nE "insmod|mknod|chmod 666|loaded OK|System initializat
ion complete" ~/embedded_lab2/rootfs/initramfs/etc/init.d/rcS
11:[ -c /dev/console ] || mknod /dev/console c 5 1
12:[ -c /dev/null ] || mknod /dev/null c 1 3
13:[ -c /dev/zero ] || mknod /dev/zero c 1 5
23:    insmod /lib/modules/5.15.0/lab2_driver.ko
26:        echo '[ LAB-02 ] lab2_driver loaded OK'
29:        mknod /dev/lab2 c 240 0
30:        chmod 666 /dev/lab2
34:        echo '[ LAB-02 ] ERROR: insmod lab2_driver failed!'
40:echo '[ LAB-02 ] System initialization complete!'
quocthinh_2303@Prestige-14-AI-Studio:~$
```

D.2. Cập nhật /etc/inittab

```
cat > ~/embedded_lab2/rootfs/initramfs/etc/inittab << 'INITTAB_EOF'
# /etc/inittab - BusyBox init configuration
# Format: <id>:<runlevel>:<action>:<process>

# Chạy rcS khi boot (sysinit)
::sysinit:/etc/init.d/rcS

# Mở shell trên console (hỏi Enter trước khi mở)
::askfirst:-/bin/sh

# Xử lý Ctrl+Alt+Del
::ctrlaltdel:/sbin/reboot

# Cleanup trước khi shutdown
::shutdown:/bin/umount -a -r
INITTAB_EOF
```

inittab

home > quocthinh_2303 > embedded_lab2 > rootfs > initramfs > etc > inittab

```
1   # /etc/inittab - BusyBox init configuration
2   # Format: <id>:<runlevel>:<action>:<process>
3
4   # Chay rcS khi boot (sysinit)
5   ::sysinit:/etc/init.d/rcS
6
7   # Mo shell trên console (hoi Enter truoc khi mo)
8   ::askfirst:-/bin/sh
9
10  # Xu ly Ctrl+Alt+Del
11  ::ctrlaltdel:/sbin/reboot
12
13  # Cleanup truoc khi shutdown
14  ::shutdown:/bin/umount -a -r
```

D.3. Kiểm tra cấu trúc rootfs trước khi đóng gói

```
cd ~/embedded_lab2/rootfs/initramfs
echo '=== Kiểm tra cấu trúc rootfs ==='
ls -la lib/modules/5.15.0/lab2_driver.ko
ls -la usr/bin/test_driver.sh usr/bin/test_procfs.sh
ls -la etc/inittab etc/init.d/rcS
```

Kiểm tra module binary đúng kiến trúc

```
file lib/modules/5.15.0/lab2_driver.ko
# Phải là ARM, EABI5 – không phải x86_64!
```

```
quocthinh_2303@Prestige-14-AI-Studio:~$ cd ~/embedded_lab2/rootfs/initramfs
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/rootfs/initramfs$ echo '=== Kiểm tra cấu trúc ro
otfs ==='
=== Kiểm tra cấu trúc rootfs ===
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/rootfs/initramfs$ ls -la lib/modules/5.15.0/lab2
_driver.ko
-rw-r--r-- 1 quocthinh_2303 quocthinh_2303 142064 Thg 10  8 22:08 lib/modules/5.15.0/lab2_driver.ko
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/rootfs/initramfs$ ls -la usr/bin/test_driver.sh
usr/bin/test_procfs.sh
-rwxrwxr-x 1 quocthinh_2303 quocthinh_2303 969 Thg 10  6 07:50 usr/bin/test_driver.sh
-rwxrwxr-x 1 quocthinh_2303 quocthinh_2303 598 Thg 10  8 22:04 usr/bin/test_procfs.sh
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/rootfs/initramfs$ ls -la etc/inittab etc/init.d/
rcS
-rwxr-xr-x 1 quocthinh_2303 quocthinh_2303 1275 Thg 10  9 06:42 etc/init.d/rcS
-rw-r--r-- 1 quocthinh_2303 quocthinh_2303  325 Thg 10  9 06:47 etc/inittab
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/rootfs/initramfs$ file lib/modules/5.15.0/lab2_d
river.ko
lib/modules/5.15.0/lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[s
ha1]=ac413d8193346c976621b13869b99803ed07ca88, with debug_info, not stripped
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/rootfs/initramfs$
```

D.4. Đóng gói initramfs và boot trên QEMU

Đóng gói lại initramfs

```
cd ~/embedded_lab2/rootfs/initramfs
find . | cpio -H newc -o | gzip >
~/embedded_lab2/output/initramfs_lab2.cpio.gz
ls -lh ~/embedded_lab2/output/initramfs_lab2.cpio.gz
```

```
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/rootfs/initramfs$ find . | cpio -H newc -o | gzi
p > ~/embedded_lab2/output/initramfs_lab2.cpio.gz
6448 blocks
quocthinh_2303@Prestige-14-AI-Studio:~/embedded_lab2/rootfs/initramfs$ ls -lh ~/embedded_lab2/output/
initramfs_lab2.cpio.gz
-rw-rw-r-- 1 quocthinh_2303 quocthinh_2303 2,2M Thg 10  9 06:50 /home/quocthinh_2303/embedded_lab2/ou
tput/initramfs_lab2.cpio.gz
```

Boot QEMU với initramfs mới

```
qemu-system-arm \
  -M vexpress-a9 \
  -cpu cortex-a9 \
  -m 512M \
  -smp 2 \
  -nographic \
  -kernel ~/embedded_lab1/output/zImage \
  -dtb ~/embedded_lab1/output/vexpress-v2p-ca9.dtb \
  -initrd ~/embedded_lab2/output/initramfs_lab2.cpio.gz \
  -append 'console=ttyAMA0,115200 rdinit=/sbin/init mem=512M'
```

Kết quả mong đợi khi boot thành công:

```
[ LAB-02 ] Starting system initialization...
[ LAB-02 ] Loading lab2_driver module...
[ LAB-02 ] lab2_driver loaded OK
[ LAB-02 ] /dev/lab2 created (major=240, minor=0)
[ LAB-02 ] System initialization complete!
/ # _
```

```
Run /sbin/init as init process
[ LAB-02 ] Starting system initialization...
[ LAB-02 ] Loading lab2_driver module...
lab2_driver: loading out-of-tree module taints kernel.
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
[ LAB-02 ] lab2_driver loaded OK
mknod: /dev/lab2: File exists
[ LAB-02 ] /dev/lab2 created (major=240, minor=0)
[ LAB-02 ] System initialization complete!
 -> Run test_driver.sh to test character device
 -> Run test_procfs.sh to test proc/sysfs interfaces
 -> cat /proc/lab2_info to view driver stats

Please press Enter to activate this console. input: ImExPS/2 Generic Explorer Mouse as /devices/platf
orm/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:motherboard-bus@40000000:iofpga@7
,00000000/10007000.kmi/serio1/input/input2
random: fast init done
```

D.5. Chạy kiểm tra toàn diện trong QEMU

Sau khi boot thành công, thực hiện các bài kiểm tra sau trong BusyBox shell:

Kiểm tra 1: Module đã load tự động

```
lsmod
# lab2_driver phải xuất hiện trong danh sách
ls -la /dev/lab2
# Device node phải đã tồn tại (tạo bởi rcS)
```

```
/ # lsmod
lab2_driver 16384 0 - Live 0x7f000000 (O)
/ # ls -la /dev/lab2
crw-rw-rw-    1 root     0         240,   0 Oct  8 23:58 /dev/lab2
/ #
```

Kiểm tra 2: Chạy test scripts

```
test_driver.sh
test_procfs.sh
```

```
/ # test_driver.sh
=== LAB-02: Device Driver Test ===
[1] Loading lab2_driver...
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
[2] Loaded modules:
lab2_driver 16384 0 - Live 0x7f000000 (O)
[3] Creating device node...
 (node already exists)
crw-------    1 root     0         240,   0 Oct  9 00:07 /dev/lab2
[4] Writing to device...
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!
]
lab2_driver: device closed
[5] Reading from device...
lab2_driver: device opened (count=2)
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
 Read back: [Hello from userspace, LAB-02!]
[6] Multiple write/read test...
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1
]
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 10 bytes to user
Message_1
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2
]
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
Message_2
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3
]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
Message_3
lab2_driver: device closed
[7] Kernel messages (dmesg):
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
=== Test PASSED ===
```

```
/ # /usr/bin/test_procfs.sh
=== Test procfs & sysfs ===
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
=====================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 0 bytes
Open count : 0
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
]
lab2_driver: device closed
--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
=====================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 30 bytes
Open count : 1
Last data : [Embedded Linux Lab2 Test Data
]
--- sysfs attributes ---
buffer_len : 30
open_count : 1
last_data  : Embedded Linux Lab2 Test Data
=== procfs/sysfs/ Test PASSED ===
/ #
```

Kiểm tra 3: Đọc /proc và /sys trực tiếp

```
cat /proc/lab2_info
ls /sys/class/lab2_class/lab2/
cat /sys/class/lab2_class/lab2/buffer_len
cat /sys/class/lab2_class/lab2/open_count
cat /sys/class/lab2_class/lab2/last_data
```

```
/ # cat /proc/lab2_info
LAB-02 Driver Statistics
=====================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 30 bytes
Open count : 1
Last data : [Embedded Linux Lab2 Test Data
]
/ # ls /sys/class/lab2_class/lab2/
buffer_len  last_data   power       uevent
dev         open_count  subsystem
/ # cat /sys/class/lab2_class/lab2/buffer_len
30
/ # cat /sys/class/lab2_class/lab2/open_count
1
/ # cat /sys/class/lab2_class/lab2/last_data
Embedded Linux Lab2 Test Data
```

Kiểm tra 4: Xem kernel log

```
dmesg | grep lab2
# Xem toàn bộ vòng đời module từ lúc load đến các thao tác read/write
```

```
/ # dmesg | grep lab2
lab2_driver: loading out-of-tree module taints kernel.
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
lab2_driver: device closed
lab2_driver: module unloaded
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!
lab2_driver: device closed
lab2_driver: device opened (count=2)
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
lab2_driver: module unloaded
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
lab2_driver: device closed
/ #
```

> 📌 **Lưu ý:** Chụp màn hình hoặc dùng script để ghi lại output của TẤT CẢ kiểm tra trên. Đây là bằng chứng bắt buộc cho báo cáo.

V. Yêu cầu nộp bài

5.1. Cấu trúc thư mục nộp bài

Nén toàn bộ thành file: MSSV_HoTen_Lab02.tar.gz với cấu trúc sau:

```
MSSV_HoTen_Lab02/
├── driver/
│   ├── lab2_driver.c          # Source code kernel module (có comment)
│   ├── Makefile               # Makefile cross-compile
│   └── lab2_driver.ko         # Module binary đã build (ARM)
├── rootfs/
│   └── initramfs/         # Toàn bộ rootfs
│       ├── etc/inittab
│       ├── etc/init.d/rcS
│       ├── lib/modules/5.15.0/lab2_driver.ko
│       └── usr/bin/test_driver.sh
├── output/
│   └── initramfs_lab2.cpio.gz # initramfs đã đóng gói, boot được
├── logs/
│   ├── boot_log.txt           # Terminal log khi boot QEMU
│   ├── driver_test.txt      # Output của test_driver.sh
│   ├── procfs_test.txt      # Output của test_procfs.sh
│   └── mtd_jffs2.txt        # Output toàn bộ phần C (trên host)
└── bao_cao/
    └── MSSV_Lab02_BaoCao.pdf
```

5.2. Yêu cầu về source code

File lab2_driver.c phải đáp ứng các yêu cầu sau:

- Có đầy đủ comment giải thích mục đích của từng hàm (open, read, write, release, init, exit).
- Có comment giải thích tại sao dùng copy_to_user() / copy_from_user() thay vì memcpy().
- Có comment giải thích cơ chế mutex và tại sao cần dùng trong driver.
- Phần procfs: comment giải thích seq_file API và hàm single_open().
- Phần sysfs: comment giải thích macro DEVICE_ATTR_RO và cơ chế kobject.

5.3. Yêu cầu báo cáo

Mục 1: Thông tin sinh viên và môi trường

- Họ và tên, MSSV, nhóm, ngày thực hiện, phiên bản kernel, phiên bản GCC toolchain.

Mục 2: Sơ đồ kiến trúc driver (BẮT BUỘC)

- Vẽ sơ đồ thể hiện: userspace → system call → VFS → file_operations → device_buffer.
- Thể hiện luồng dữ liệu khi write: user buffer → copy_from_user → device_buffer → procfs/sysfs.

- Thể hiện cấu trúc /sys/class/lab2_class/lab2/ với các attribute file.

Mục 3: Log thực hành (screenshot hoặc text log)

- Phần A: output modinfo, insmod, lsmod, kết quả đọc/ghi /dev/lab2, dmesg | grep lab2.
- Phần B: output cat /proc/lab2_info (trước và sau write), toàn bộ các sysfs attributes.
- Phần C: output mtdinfo, flash_erase, nandwrite, mount JFFS2, đọc file, kiểm tra persistence sau umount/mount.
- Phần D: boot log QEMU thể hiện auto-load module, toàn bộ output của test_driver.sh và test_procfs.sh.

Mục 4: So sánh filesystem (bảng so sánh)

- Lập bảng so sánh ít nhất 5 tiêu chí: writable, wear leveling, compression, boot time, phù hợp thiết bị.
- Đưa ra khuyến nghị: dùng filesystem nào cho một thiết bị IoT có 32MB NAND flash và cần update firmware OTA?

Mục 5: Trả lời câu hỏi phân tích (Mục VI)

Mục 6: Nhận xét & Khó khăn

- Mô tả ít nhất 2 lỗi/khó khăn gặp phải và cách giải quyết.

5.4. Tiêu chí đánh giá

#	Tiêu chí đánh giá	Điểm tối đa	Mô tả
1	Phần A – Kernel Module hoạt động đúng	25	Module nạp/gỡ được, read/write hoạt động qua /dev
2	Phần B – Giao tiếp sysfs/procfs	20	Xuất thông tin qua /proc và /sys, đọc được từ userspace
3	Phần C – MTD & JFFS2 hoạt động	25	Format JFFS2 thành công, mount, read/write, kiểm chứng persistence
4	Phần D – BusyBox init tự động load module	15	Module được load tự động khi boot qua rcS script
5	Báo cáo & Phân tích câu hỏi	15	Giải thích kỹ thuật chính xác, so sánh filesystem đúng
	Tổng cộng	**100**	

VI. Câu hỏi phân tích

Trả lời chi tiết các câu hỏi sau trong báo cáo. Mỗi câu hỏi 3 điểm trong tổng 15 điểm mục này:

Câu 1. Tại sao không thể dùng memcpy() để sao chép dữ liệu giữa kernel buffer và user buffer trong hàm lab2_read() / lab2_write()? Giải thích khái niệm kernel space và user space address, và cơ chế mà copy_to_user() / copy_from_user() xử lý địa chỉ ảo khác nhau này. Điều gì xảy ra nếu dùng memcpy() thay thế trong kernel?

Trả lời: Trong Linux, chương trình người dùng và kernel hoạt động ở hai không gian bộ nhớ khác nhau là **user space** và **kernel space**. Vì vậy, con trỏ mà chương trình userspace truyền vào driver không thể được sử dụng giống như một con trỏ bình thường bên trong kernel.

Nếu dùng memcpy() trực tiếp để đọc hoặc ghi vào vùng nhớ của userspace, kernel không có cơ chế kiểm tra xem địa chỉ đó có hợp lệ và có thể truy cập được hay không. Nếu địa chỉ không hợp lệ, việc truy cập có thể gây lỗi và ảnh hưởng đến kernel.

Do đó, trong driver của bài LAB2, em sử dụng copy_from_user() khi cần lấy dữ liệu từ userspace vào device_buffer, và copy_to_user() khi trả dữ liệu từ device_buffer về cho chương trình người dùng. Hai hàm này được kernel cung cấp riêng cho việc trao đổi dữ liệu giữa hai không gian bộ nhớ và có cơ chế xử lý trường hợp truy cập thất bại. Trong phần thực hành, khi ghi chuỗi vào /dev/lab2 rồi đọc lại, dữ liệu nhận được vẫn đúng với dữ liệu ban đầu, cho thấy quá trình trao đổi giữa userspace và kernel space đã hoạt động đúng.

Câu 2. Driver dùng DEFINE_MUTEX(lab2_mutex) để bảo vệ device_buffer. Giải thích tại sao cần mutex trong kernel driver khi có nhiều process cùng mở /dev/lab2. Nếu không có mutex, điều gì có thể xảy ra khi hai process đồng thời gọi write()? Mô tả race condition cụ thể và hậu quả.

Trả lời: Trong driver, device_buffer, buffer_len và một số biến trạng thái được dùng chung cho tất cả process mở /dev/lab2. Nếu có hai process cùng ghi dữ liệu vào driver tại gần như cùng một thời điểm mà không có cơ chế đồng bộ, các thao tác của hai process có thể xen kẽ nhau và tạo ra **race condition** (là lỗi xảy ra khi nhiều luồng cùng truy cập dữ liệu dùng chung mà không được đồng bộ).

Ví dụ: process thứ nhất đang ghi một chuỗi vào device_buffer, nhưng chưa hoàn thành thì process thứ hai cũng bắt đầu ghi. Khi đó nội dung buffer hoặc giá trị buffer_len có thể không còn đồng nhất. Kết quả là dữ liệu đọc lại có thể bị sai, bị ghi đè hoặc không đúng với dữ liệu mà process cuối cùng đã gửi.

Để tránh tình trạng này, driver sử dụng DEFINE_MUTEX(lab2_mutex). Trước khi thay đổi dữ liệu dùng chung, driver gọi mutex_lock(), và sau khi thao tác hoàn tất thì gọi mutex_unlock(). Nhờ đó, tại một thời điểm chỉ có một luồng được phép thay đổi vùng dữ liệu được bảo vệ. Trong bài test, em thực hiện nhiều lần ghi và đọc với Message_1, Message_2, Message_3 và dữ liệu đều được đọc lại đúng, cho thấy trạng thái của buffer được duy trì ổn định.

Câu 3. So sánh hai cơ chế cung cấp thông tin từ kernel ra userspace: procfs (/proc) và sysfs (/sys). Tại sao kernel documentation khuyến nghị không thêm file mới vào /proc mà nên dùng /sys thay thế? Trong driver của bài lab, thông tin nào phù hợp đặt trong /proc, thông tin nào phù hợp trong /sys và tại sao?

Trả lời: Cả procfs và sysfs đều là các filesystem ảo, nghĩa là dữ liệu trong đó phản ánh trạng thái hiện tại của kernel chứ không phải dữ liệu được lưu cố định trên ổ đĩa. Tuy nhiên, cách tổ chức và mục đích sử dụng của hai cơ chế này khác nhau.

Trong LAB2, /proc/lab2_info phù hợp để hiển thị **một nhóm thông tin tổng hợp** về driver. Chỉ cần dùng một lệnh cat, em có thể xem cùng lúc tên driver, major number, kích thước buffer, số byte dữ liệu hiện tại, số lần mở device và dữ liệu cuối cùng được ghi.

Ngược lại, sysfs chia từng thuộc tính thành các file riêng, ví dụ buffer_len, open_count và last_data. Cách tổ chức này rõ ràng hơn khi thông tin gắn trực tiếp với một device và cũng thuận tiện cho script hoặc chương trình userspace chỉ cần đọc một thuộc tính cụ thể.

Vì vậy, trong bài này em xem procfs như một nơi hiển thị **thống kê tổng quan**, còn sysfs dùng để biểu diễn **từng thuộc tính riêng của device**. Sau khi ghi dữ liệu vào /dev/lab2, cả /proc/lab2_info và các file trong /sys/class/lab2_class/lab2/ đều cập nhật đúng trạng thái của driver. Đề LAB cũng yêu cầu kiểm tra trực tiếp cả hai giao diện này.

Câu 4. Tại sao JFFS2 phải xóa (erase) toàn bộ erase block trước khi có thể ghi dữ liệu mới, trong khi ext4 có thể ghi trực tiếp lên bất kỳ sector nào? Giải thích đặc tính vật lý của NAND flash dẫn đến yêu cầu này. Cơ chế log-structured của JFFS2 giải quyết vấn đề này như thế nào?

Trả lời: Raw NAND flash có đặc tính vật lý khác với các thiết bị block như HDD, SD card hoặc eMMC. Trên NAND flash, dữ liệu không thể được ghi đè tùy ý lên vùng đã có dữ liệu. Trước khi ghi lại, vùng flash phải được đưa về trạng thái đã xóa bằng thao tác **erase**, và thao tác này được thực hiện theo từng **erase block** chứ không phải từng byte riêng lẻ.

Đó là lý do trong phần thực hành em phải chạy flash_erase trước khi dùng nandwrite để ghi image JFFS2 vào NAND simulator. Tài liệu LAB cũng yêu cầu thực hiện đúng thứ tự này trước khi ghi filesystem image.

JFFS2 được thiết kế riêng cho raw flash nên không cố ghi đè dữ liệu cũ theo cách của filesystem thông thường. Thay vào đó, nó hoạt động theo kiểu **log-structured**, tức là ghi phiên bản dữ liệu mới vào vùng phù hợp và sau đó thu hồi những vùng dữ liệu cũ không còn cần thiết. Cách làm này phù hợp với đặc tính erase-before-write của flash, đồng thời hỗ trợ việc phân bố số lần ghi/xóa giữa các block.

Trong bài thực hành, sau khi em tạo file mới trên JFFS2, unmount filesystem rồi mount lại, file vẫn đọc được với nội dung New runtime file. Kết quả này cho thấy dữ liệu đã được ghi thực sự xuống NAND simulator và có tính persistence.

Câu 5. Trong quá trình boot, BusyBox init đọc /etc/inittab và chạy rcS. Giải thích: nếu lệnh insmod trong rcS thất bại (module bị lỗi hoặc không tìm thấy), hệ thống có tiếp tục boot không? Làm thế nào để rcS phân biệt lỗi nghiêm trọng (dừng boot) với lỗi không nghiêm trọng (tiếp tục boot với cảnh báo)? Cải thiện rcS script của bài lab để xử lý tốt hơn trường hợp module load thất bại.

Trả lời: Khi BusyBox khởi động, /etc/inittab gọi /etc/init.d/rcS ở giai đoạn sysinit. Trong LAB2, rcS được dùng để mount các pseudo filesystem, thiết lập hệ thống và tự động load lab2_driver.ko.

Nếu lệnh insmod thất bại thì hệ thống **không nhất thiết phải dừng boot**. Điều đó phụ thuộc vào cách script xử lý lỗi. Trong rcS của bài LAB, sau khi chạy insmod, script kiểm tra giá trị trả về. Nếu module load thành công thì tiếp tục tạo /dev/lab2; nếu thất bại thì chỉ in ra thông báo lỗi. Sau đó script vẫn tiếp tục đến phần System initialization complete, vì vậy BusyBox vẫn có thể mở shell.

Theo em, cách xử lý nên phụ thuộc vào mức độ quan trọng của module. Nếu module chỉ cung cấp một chức năng bổ sung thì có thể xem lỗi này là không nghiêm trọng, in cảnh báo rồi cho hệ thống tiếp tục boot. Ngược lại, nếu thiết bị không thể hoạt động an toàn nếu thiếu driver thì đây nên được xem là lỗi nghiêm trọng và script cần dừng hoặc chuyển hệ thống sang chế độ recovery.

Có thể viết rcS rõ ràng hơn theo dạng:

```
if insmod /lib/modules/5.15.0/lab2_driver.ko; then
    echo '[ LAB-02 ] lab2_driver loaded OK'
    mknod /dev/lab2 c 240 0 2>/dev/null || true
    chmod 666 /dev/lab2
else
    echo '[ LAB-02 ] WARNING: failed to load lab2_driver'
    echo '[ LAB-02 ] Continuing boot without /dev/lab2'
fi
```

Trong lúc thực hành, module đã được load thành công khi boot và log hiển thị lab2_driver loaded OK, nên trường hợp lỗi trên chủ yếu là tình huống cần dự phòng khi thiết kế script khởi động.

Khó khăn & Nhận xét

1. Những khó khăn gặp phải

- Module đã được load trước khi chạy test script

Trong quá trình thực hành, em gặp lỗi File exists khi chạy test_driver.sh và test_procfs.sh. Nguyên nhân là module lab2_driver đã được tự động nạp trong quá trình khởi động thông qua script rcS, trong khi các script kiểm tra vẫn tiếp tục thực hiện lệnh insmod.

Để khắc phục, em kiểm tra trạng thái module bằng lsmod, sau đó sử dụng rmmod lab2_driver để gỡ module trước khi chạy lại các bài test. Sau khi xử lý, cả hai script đều thực hiện thành công và hiển thị kết quả Test PASSED.

- Xác định đúng thiết bị NAND simulator

Khi thực hiện phần MTD/JFFS2, em nhận thấy NAND simulator xuất hiện tại /dev/mtd1 thay vì /dev/mtd0 như ví dụ trong tài liệu. Nguyên nhân là /dev/mtd0 đã được hệ thống sử dụng cho thiết bị có tên BIOS.

Để tránh thao tác nhầm thiết bị, em sử dụng cat /proc/mtd và mtdinfo để xác định chính xác NAND simulator trước khi thực hiện các lệnh flash_erase, nandwrite và mount JFFS2. Kết quả cho thấy filesystem hoạt động bình thường và dữ liệu vẫn được giữ lại sau khi unmount rồi mount lại.

2. Nhận xét chung

Qua bài LAB2, em hiểu rõ hơn cách xây dựng và kiểm tra một character device driver trên Embedded Linux, đặc biệt là việc trao đổi dữ liệu giữa userspace và kernel space thông qua các thao tác đọc, ghi. Bên cạnh đó, việc sử dụng procfs và sysfs giúp em hiểu cách kernel cung cấp thông tin trạng thái cho chương trình người dùng.

Phần thực hành MTD/JFFS2 và tích hợp module vào BusyBox init cũng giúp em hiểu thêm về cách quản lý filesystem trên NAND flash và quá trình khởi động của hệ thống nhúng. Qua những lỗi gặp phải, em nhận thấy việc kiểm tra log và trạng thái thực tế của hệ thống là rất quan trọng để xác định nguyên nhân và xử lý lỗi chính xác.

VII. Phụ lục – Tài liệu tham khảo & Debug

7.1. Tài liệu tham khảo

- Embedded Linux Primer (2nd Ed.) – Christopher Hallinan, Chapters 8, 9, 10, 11
- Linux Device Drivers (3rd Ed.) – Corbet, Rubini, Kroah-Hartman (free: lwn.net/Kernel/LDD3)
- The Linux Kernel Module Programming Guide: tldp.org/LDP/lkmpg/2.6/html/
- Linux Kernel Documentation – driver-api/: kernel.org/doc/html/latest/driver-api/
- MTD Linux Documentation: linux-mtd.infradead.org/doc/
- JFFS2 design document: linux-mtd.infradead.org/tech/jffs2.html

7.2. Lỗi thường gặp và cách xử lý

Thông báo lỗi	Nguyên nhân	Giải pháp
`insmod: ERROR: could not insert module`	vermagic không khớp với kernel target	`Build module với đúng kernel source của LAB-01`
`FATAL: Module lab2_driver not found`	Sai đường dẫn hoặc file không tồn tại	Kiểm tra ls /lib/modules/5.15.0/lab2_driver.ko trong QEMU
`mknod: /dev/lab2: File exists`	Device node đã tồn tại từ trước	`Xóa trước: rm /dev/lab2, sau đó mknod lại`
`open /dev/lab2: Permission denied`	Quyền truy cập không đủ	chmod 666 /dev/lab2 hoặc chạy với root
`make: /lib/modules/.../build: No such file`	Thiếu kernel headers trên host	`Trỏ KDIR đến kernel source của LAB-01`
`error: implicit declaration of function 'class_create'`	Kernel API thay đổi từ v6.4+	Dùng kernel 5.15 LTS như hướng dẫn
`flash_erase: /dev/mtd0: No such file`	nandsim chưa được load	sudo modprobe nandsim trước
`mount: /dev/mtdblock0: Invalid argument`	Filesystem chưa được format đúng	Kiểm tra lại mkfs.jffs2 --eraseblock khớp với mtdinfo

7.3. Lệnh debug hữu ích

Theo dõi kernel log realtime

```
dmesg -w | grep lab2
# -w: watch mode, tự động cập nhật khi có log mới
```

Xem tất cả character devices đã đăng ký

```
cat /proc/devices | grep -A20 'Character'
# Kiểm tra major number 240 đã được lab2 đăng ký chưa
```

Xem cây kobject trong sysfs

```
find /sys/class/lab2_class/ -type f
# Liệt kê tất cả attribute files trong sysfs
```

Kiểm tra nội dung initramfs

```
zcat ~/embedded_lab2/output/initramfs_lab2.cpio.gz | cpio -t | grep -E
'lab2|modules'
# Xác nhận module và scripts đã vào trong initramfs
```

Xem MTD partition layout

```
cat /proc/mtd
mtdinfo -a
# -a: hiển thị thông tin tất cả MTD devices
```

Debug khi QEMU không hiển thị output

```
# Thêm -serial mon:stdio vào lệnh QEMU
qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M \
  -nographic -serial mon:stdio \
  -kernel ... -dtb ... -initrd ...  \
  -append 'console=ttyAMA0,115200 root=/dev/ram0 rw'
```

7.4. Cấu trúc kernel module – Sơ đồ tham khảo

```
insmod lab2_driver.ko
    ↓
lab2_init() ──► register_chrdev(240, "lab2", &fops)
     ──► class_create() → lab2_class
     ──► device_create() → /sys/class/lab2_class/lab2
     ──► proc_create() → /proc/lab2_info
    ↓
User: open("/dev/lab2") ──► lab2_open()
User: write(fd, buf, n) ──► lab2_write() ──► device_buffer
User: read(fd, buf, n)  ──► lab2_read()  ──► device_buffer
User: close(fd)         ──► lab2_release()
    ↓
rmmod lab2_driver → lab2_exit() ──► cleanup all resources
```

Bài lab này là một phần của khóa học Embedded Linux Systems. Mọi thắc mắc vui lòng liên hệ giảng viên phụ trách trong giờ hành chính hoặc qua hệ thống LMS của trường.